

```

/*
=====
=
    Interest Calculator – Logic
    Rule: Principal is updated with interest
only
    once, after the first complete 3-year
block.
    Everything beyond 3 years is calculated
in one
    shot on that updated principal. No
further
    compounding happens inside the remaining
period.

=====
= */

(function () {
    "use strict";

    const principalInput =
document.getElementById("principal");
    const rateInput =
document.getElementById("rate");
    const yearsInput =
document.getElementById("years");
    const monthsInput =
document.getElementById("months");
    const daysInput =

```

```
document.getElementById("days");

    const calculateBtn =
document.getElementById("calculateBtn");
    const resetBtn =
document.getElementById("resetBtn");
    const errorMessage =
document.getElementById("errorMessage");
    const resultsPanel =
document.getElementById("resultsPanel");

    const finalTotalEl =
document.getElementById("finalTotal");
    const outPrincipal =
document.getElementById("outPrincipal");
    const outRate =
document.getElementById("outRate");
    const outTenure =
document.getElementById("outTenure");
    const outTotalInterest =
document.getElementById("outTotalInterest")
;
    const breakdownEl =
document.getElementById("breakdown");

    const BLOCK_YEARS = 3;

    function formatINR(amount) {
        const value = Number(amount) || 0;
        return "₹" + value.toLocaleString("en-
IN", {
            maximumFractionDigits: 2,
            minimumFractionDigits: 2
        });
    };

```

```

    }

    function formatTenure(years, months,
days) {
        const parts = [];
        if (years) parts.push(years + (years
=== 1 ? " Year" : " Years"));
        if (months) parts.push(months + (months
=== 1 ? " Month" : " Months"));
        if (days) parts.push(days + (days === 1
? " Day" : " Days"));
        return parts.length ? parts.join(" ") :
"0 Days";
    }

    function showError(message) {
        errorMessage.textContent = message;
        errorMessage.classList.add("visible");
        resultsPanel.hidden = true;
    }

    function clearError() {
        errorMessage.textContent = "";

errorMessage.classList.remove("visible");
    }

    function readInputs() {
        const principal =
parseFloat(principalInput.value);
        const rate =
parseFloat(rateInput.value);
        const years = parseInt(yearsInput.value
|| "0", 10);

```

```

    const months =
parseInt(monthsInput.value || "0", 10);
    const days = parseInt(daysInput.value
|| "0", 10);
    return { principal, rate, years,
months, days };
}

function validate({ principal, rate,
years, months, days }) {
    if (isNaN(principal) || principal <= 0)
{
        return "Please enter a valid
Principal Amount greater than 0.";
    }
    if (isNaN(rate) || rate < 0) {
        return "Please enter a valid Interest
Rate.";
    }
    if ([years, months, days].some((v) =>
isNaN(v) || v < 0)) {
        return "Tenure values cannot be
negative.";
    }
    if (months > 11) {
        return "Months should be between 0
and 11.";
    }
    if (days > 30) {
        return "Days should be between 0 and
30.";
    }
    if (years === 0 && months === 0 && days
=== 0) {

```

```

        return "Please enter a tenure greater
than 0.";
    }
    return null;
}

function timeInYears(years, months, days)
{
    return years + months / 12 + days /
365;
}

function calculate({ principal, rate,
years, months, days }) {
    const totalTenureYears =
timeInYears(years, months, days);

    if (totalTenureYears <= BLOCK_YEARS) {
        // Tenure is within the first block –
calculate directly, no principal update.
        const interest = (principal * rate *
totalTenureYears) / 100;
        const finalTotal = principal +
interest;

        return {
            usesBlock: false,
            principal,
            rate,
            totalTenureYears,
            tenureLabel: formatTenure(years,
months, days),
            directInterest: interest,
            finalTotal,

```

```
        totalInterest: interest
    };
}

// Step 1: first complete 3-year block
const firstBlockInterest = (principal *
rate * BLOCK_YEARS) / 100;
const updatedPrincipal = principal +
firstBlockInterest;

// Step 2: remaining tenure, calculated
in one shot on the updated principal
const remainingYears = years -
BLOCK_YEARS;
const remainingMonths = months;
const remainingDays = days;
const remainingTenureYears =
timeInYears(remainingYears,
remainingMonths, remainingDays);

const remainingInterest =
(updatedPrincipal * rate *
remainingTenureYears) / 100;
const finalTotal = updatedPrincipal +
remainingInterest;
const totalInterest = finalTotal -
principal;

return {
    usesBlock: true,
    principal,
    rate,
    totalTenureYears,
    tenureLabel: formatTenure(years,
```

```

months, days),
    firstBlockInterest,
    updatedPrincipal,
    remainingYears,
    remainingMonths,
    remainingDays,
    remainingTenureYears,
    remainingTenureLabel:
formatTenure(remainingYears,
remainingMonths, remainingDays),
    remainingInterest,
    finalTotal,
    totalInterest
    };
}

function renderBreakdown(result) {
    breakdownEl.innerHTML = "";

    const steps = [];

    if (!result.usesBlock) {
        steps.push({
            title: "Direct Calculation (Tenure
≤ 3 Years)",
            lines: [
                `Interest = Principal × Rate ×
Time / 100`,
                `Interest =
${formatINR(result.principal)} ×
${result.rate}% ×
${result.totalTenureYears.toFixed(4)} yrs /
100`,
                `Interest =

```

```

<strong>${formatINR(result.directInterest)}
</strong>`
    ]
    });
    steps.push({
        title: "Final Total",
        lines: [
            `Final Total = Principal +
Interest`,
            `Final Total =
${formatINR(result.principal)} +
${formatINR(result.directInterest)}`,
            `Final Total =
<strong>${formatINR(result.finalTotal)}
</strong>`
        ]
    });
} else {
    steps.push({
        title: "Step 1 – First 3 Years",
        lines: [
            `Interest = Principal × Rate × 3
/ 100`,
            `Interest =
${formatINR(result.principal)} ×
${result.rate}% × 3 / 100`,
            `Interest for first 3 years =
<strong>${formatINR(result.firstBlockIntere
st)}</strong>`
        ]
    });
    steps.push({
        title: "Updated Principal",
        lines: [

```



```

        `Updated Principal = Original
Principal + First 3-Year Interest`,
        `Updated Principal =
${formatINR(result.principal)} +
${formatINR(result.firstBlockInterest)}`,
        `Updated Principal =
<strong>${formatINR(result.updatedPrincipal
)}</strong>`
    ]
    });
    steps.push({
        title: `Step 2 – Remaining
${result.remainingTenureLabel}`,
        lines: [
            `Remaining Time = Years + (Months
/ 12) + (Days / 365)`,
            `Remaining Time =
${result.remainingYears} +
(${result.remainingMonths} / 12) +
(${result.remainingDays} / 365) =
${result.remainingTenureYears.toFixed(4)}
yrs`,
            `Remaining Interest = Updated
Principal × Rate × Remaining Time / 100`,
            `Remaining Interest =
${formatINR(result.updatedPrincipal)} ×
${result.rate}% ×
${result.remainingTenureYears.toFixed(4)} /
100`,
            `Remaining Interest =
<strong>${formatINR(result.remainingInteres
t)}</strong>`
        ]
    });

```

```

        steps.push({
            title: "Final Total",
            lines: [
                `Final Total = Updated Principal
+ Remaining Interest`,
                `Final Total =
${formatINR(result.updatedPrincipal)} +
${formatINR(result.remainingInterest)}`,
                `Final Total =
<strong>${formatINR(result.finalTotal)}
</strong>`
            ]
        });
    }

    steps.forEach((step) => {
        const stepEl =
document.createElement("div");
        stepEl.className = "step";

        const titleEl =
document.createElement("div");
        titleEl.className = "step-title";
        titleEl.textContent = step.title;
        stepEl.appendChild(titleEl);

        step.lines.forEach((line) => {
            const lineEl =
document.createElement("div");
            lineEl.className = "step-line";
            lineEl.innerHTML = line;
            stepEl.appendChild(lineEl);
        });
    });

```

```

        breakdownEl.appendChild(stepEl);
    });
}

function renderResults(result) {
    finalTotalEl.textContent =
formatINR(result.finalTotal);
    outPrincipal.textContent =
formatINR(result.principal);
    outRate.textContent = result.rate + "%
/ year";
    outTenure.textContent =
result.tenureLabel;
    outTotalInterest.textContent =
formatINR(result.totalInterest);

    renderBreakdown(result);

    resultsPanel.hidden = false;
}

function handleCalculate() {
    const inputs = readInputs();
    const errorText = validate(inputs);

    if (errorText) {
        showError(errorText);
        return;
    }

    clearError();
    const result = calculate(inputs);
    renderResults(result);
    resultsPanel.scrollToView({ behavior:

```

```
"smooth", block: "start" });  
}  
  
function handleReset() {  
  principalInput.value = "";  
  rateInput.value = "";  
  yearsInput.value = "";  
  monthsInput.value = "";  
  daysInput.value = "";  
  clearError();  
  resultsPanel.hidden = true;  
  breakdownEl.innerHTML = "";  
}  
  
calculateBtn.addEventListener("click",  
handleCalculate);  
resetBtn.addEventListener("click",  
handleReset);  
  
[principalInput, rateInput, yearsInput,  
monthsInput, daysInput].forEach((el) => {  
  el.addEventListener("keydown", (e) => {  
    if (e.key === "Enter") {  
      handleCalculate();  
    }  
  });  
});  
})();
```